

DECODELABS INDUSTRIAL TRAINING PROGRAM 2026

FULL STACK PROJECT 2 BACKEND API DEVELOPMENT

SUBMITTED BY LAIBA ZAHID

1. EXECUTIVE SUMMARY

This project presents the development of a RESTful API-based Student Attendance Management System using Node.js and Express.js. The application enables attendance record management through API endpoints that support Create, Read, and Delete operations.

The system incorporates server-side validation, proper HTTP status codes, structured JSON responses, and comprehensive API testing through Postman. The project demonstrates modern backend development practices and follows industry-standard API design principles.

2. PROJECT INFORMATION

2.1 Project Details

Item	Details
Project Name	Full Stack Project 2 – Backend API Development
Domain	Backend Development / API
Platform	Node.js + Express.js

3. PROJECT DESCRIPTION

The Student Attendance Management API is a RESTful backend service developed using Node.js and Express.js. The API allows users to manage student attendance records efficiently through standardized HTTP methods.

The application performs:

- Attendance record creation
- Attendance retrieval
- Individual student record retrieval
- Record deletion
- Data validation

- Error handling
- JSON response generation

4. PROJECT OBJECTIVES

The primary objectives of the project are:

1. Build RESTful API endpoints using Express.js.
2. Implement server-side validation.
3. Handle user input securely.
4. Return structured JSON responses.
5. Use proper HTTP status codes.
6. Demonstrate API testing using Postman.
7. Implement error handling mechanisms.

5. KEY FEATURES

Feature	Description
GET /attendance	Fetch all attendance records
GET /attendance/:id	Fetch single student attendance
POST /attendance	Mark attendance
DELETE /attendance/:id	Delete attendance record
Validation	Name, Roll Number, Date, Status
Status Codes	200, 201, 400, 404, 500
JSON Responses	Structured API responses

6. REQUIREMENTS CHECKLIST

6.1 Requirement Verification

#	Requirement	Status	Evidence
1	GET Endpoint	Completed	/attendance
2	POST Endpoint	Completed	/attendance
3	Handle User Input	Completed	req.body parsing
4	Data Validation	Completed	Name, Roll, Date, Status
5	Proper Responses	Completed	JSON Messages
6	HTTP Status Codes	Completed	200, 201, 400, 404, 500
7	JSON Data Format	Completed	application/json
8	Server-Side Logic	Completed	Express.js
9	API Testing	Completed	Postman
10	Error Handling	Completed	try-catch blocks

6.2 Completion Metrics

Metric	Value
Total Requirements	10
Completed Requirements	10
Completion Percentage	100%

7. TECHNOLOGY STACK

7.1 Backend Technologies

Technology	Purpose
Node.js	JavaScript Runtime Environment

Technology	Purpose
Express.js	Backend Web Framework
JavaScript	Programming Language
JSON	Data Exchange Format

7.2 Tools and Services

Tool	Purpose
Postman	API Testing
Visual Studio Code	Development Environment
GitHub	Version Control
Command Prompt	Server Management

8. API ENDPOINTS DOCUMENTATION

8.1 Endpoint Overview

Method	Endpoint	Description	Status Codes
GET	/	Welcome Message	200
GET	/attendance	Get All Attendance Records	200, 500
GET	/attendance/:id	Get Single Student	200, 404, 500
POST	/attendance	Mark Attendance	201, 400, 500
DELETE	/attendance/:id	Delete Student Record	200, 404, 500

9. REQUEST AND RESPONSE EXAMPLES

9.1 GET Attendance Records

Request

GET http://localhost:3000/attendance

Response (200 OK)

```
{
  "success": true,
  "count": 2,
  "data": [
    {
      "id": 1,
      "name": "Ahmad Ali",
      "rollNumber": "2024-CS-01",
      "date": "2026-06-17",
      "status": "present"
    }
  ]
}
```

9.2 POST Attendance Record

Request

POST http://localhost:3000/attendance

Request Body

```
{
  "name": "Usman Raza",
  "rollNumber": "2024-CS-03",
  "date": "2026-06-17",
  "status": "present"
}
```

Response (201 Created)

```
{
  "success": true,
  "message": "Attendance marked successfully!",
  "data": {

```

```
{
  "id": 3,
  "name": "Usman Raza",
  "rollNumber": "2024-CS-03",
  "date": "2026-06-17",
  "status": "present"
}
```

10. DATA VALIDATION RULES

Field	Validation Rule	Error Message
Name	Required and Non-empty	Student name is required
Roll Number	Required and Non-empty	Roll number is required
Date	Required in YYYY-MM-DD format	Date must be in YYYY-MM-DD format
Status	Must be Present or Absent	Status must be either present or absent

11. HTTP STATUS CODES

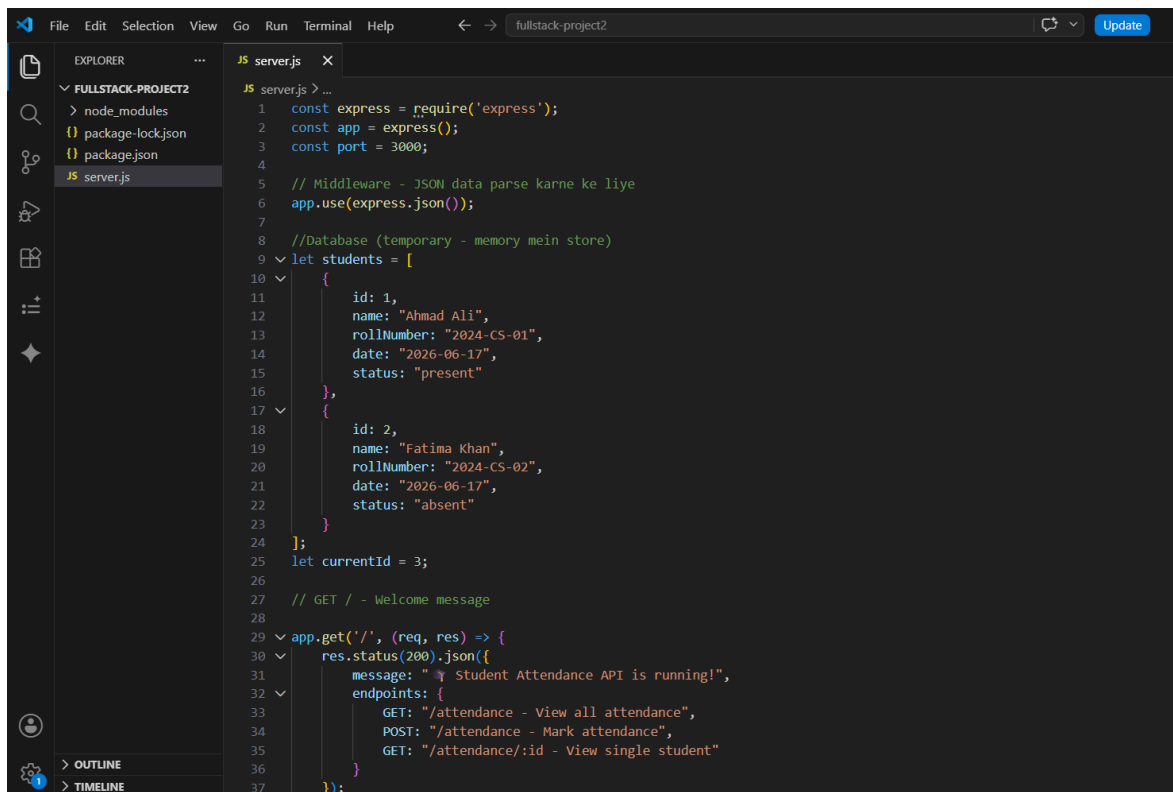
Code	Meaning	Usage
200	OK	Successful GET requests
201	Created	Record created successfully
400	Bad Request	Validation failure
404	Not Found	Student ID not found
500	Internal Server Error	Unexpected server error

12. PROJECT FILE STRUCTURE

```
fullstack-project2/  
├── node_modules/  
├── package.json  
├── package-lock.json  
└── server.js
```

13. SCREENSHOTS DOCUMENTATION

Screenshot 1 – VS Code Project Structure



The screenshot shows the VS Code interface with the Explorer sidebar on the left and the editor window on the right. The Explorer sidebar shows the project structure for 'fullstack-project2', including 'node_modules', 'package-lock.json', 'package.json', and 'server.js'. The editor window displays the contents of 'server.js', which is a JavaScript file for an Express.js application. The code includes imports for 'express', middleware for JSON parsing, a temporary in-memory database of students, and a GET endpoint for a welcome message.

```
1  const express = require('express');  
2  const app = express();  
3  const port = 3000;  
4  
5  // Middleware - JSON data parse karne ke liye  
6  app.use(express.json());  
7  
8  //Database (temporary - memory mein store)  
9  let students = [  
10   {  
11     id: 1,  
12     name: "Ahmad Ali",  
13     rollNumber: "2024-CS-01",  
14     date: "2026-06-17",  
15     status: "present"  
16   },  
17   {  
18     id: 2,  
19     name: "Fatima Khan",  
20     rollNumber: "2024-CS-02",  
21     date: "2026-06-17",  
22     status: "absent"  
23   }  
24 ];  
25 let currentId = 3;  
26  
27 // GET / - Welcome message  
28  
29 app.get('/', (req, res) => {  
30   res.status(200).json({  
31     message: "Student Attendance API is running!",  
32     endpoints: {  
33       GET: "/attendance - View all attendance",  
34       POST: "/attendance - Mark attendance",  
35       GET: "/attendance/:id - View single student"  
36     }  
37   });  
38 });
```


Screenshot 2 – Server Running

```
Administrator: Command Prompt - node server.js
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\Users\laiba\OneDrive\Desktop\fullstack-project2

C:\Users\laiba\OneDrive\Desktop\fullstack-project2>node server.js
Attendance API is running at http://localhost:3000
GET → http://localhost:3000/attendance
POST → http://localhost:3000/attendance
```

Screenshot 3 – GET Request (All Students)

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/attendance`
- Method:** `GET`
- Status:** `200 OK` (6 ms, 458 B)
- Body (JSON):**

```
{
  "success": true,
  "count": 2,
  "data": [
    {
      "id": 1,
      "name": "Ahmad Ali",
      "rollNumber": "2024-CS-01",
      "date": "2026-06-17",
      "status": "present"
    },
    {
      "id": 2,
      "name": "Fatima Khan",
      "rollNumber": "2024-CS-02",
      "date": "2026-06-17",
      "status": "absent"
    }
  ]
}
```

	id	name	rollNumber	date	status
0	1	Ahmad Ali	2024-CS-01	2026-06-17	present
1	2	Fatima Khan	2024-CS-02	2026-06-17	absent

Screenshot 4 – GET Request (Single Student)

The screenshot shows a REST client interface with a workspace named 'Laiba Zahid's Workspace'. The active request is a GET request to 'http://localhost:3000/attendance/1'. The response status is '200 OK' with a response time of 5 ms and a body size of 352 B. The response body is in JSON format, showing a successful result with a single student record.

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Ahmad Ali",
      "rollNumber": "2024-CS-01",
      "date": "2026-06-17",
      "status": "present"
    }
  ]
}
```

Screenshot 5 – POST Request (Valid Data)

The screenshot shows a REST client interface with a workspace named 'Laiba Zahid's Workspace'. The active request is a POST request to 'http://localhost:3000/attendance'. The response status is '201 Created' with a response time of 5 ms and a body size of 402 B. The response body is in JSON format, showing a successful result with a new student record.

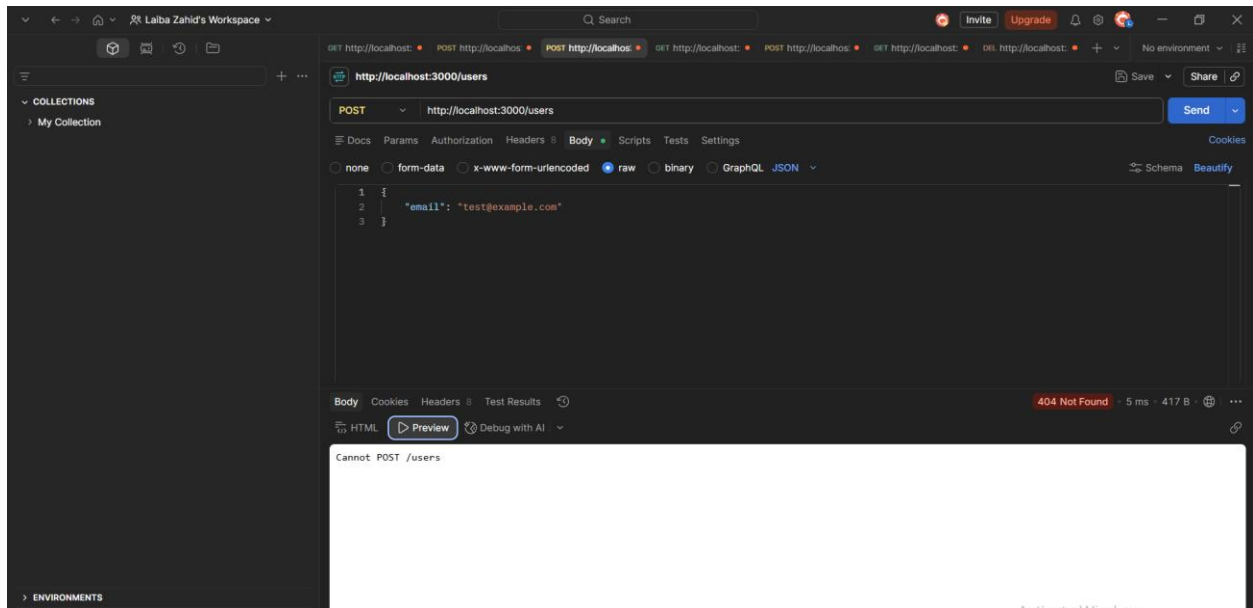
Body

```
{
  "name": "Usman Raza",
  "rollNumber": "2024-CS-03",
  "date": "2026-06-17",
  "status": "present"
}
```

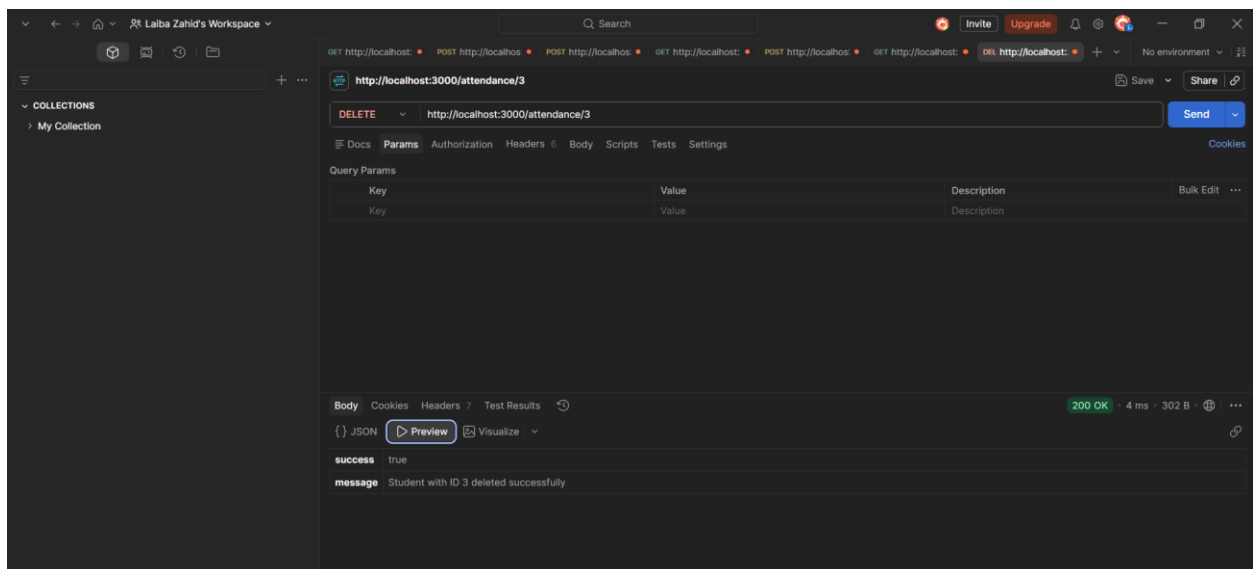
Body

```
{
  "success": true,
  "message": "Attendance marked successfully!",
  "data": [
    {
      "id": 3,
      "name": "Usman Raza",
      "rollNumber": "2024-CS-03",
      "date": "2026-06-17",
      "status": "present"
    }
  ]
}
```

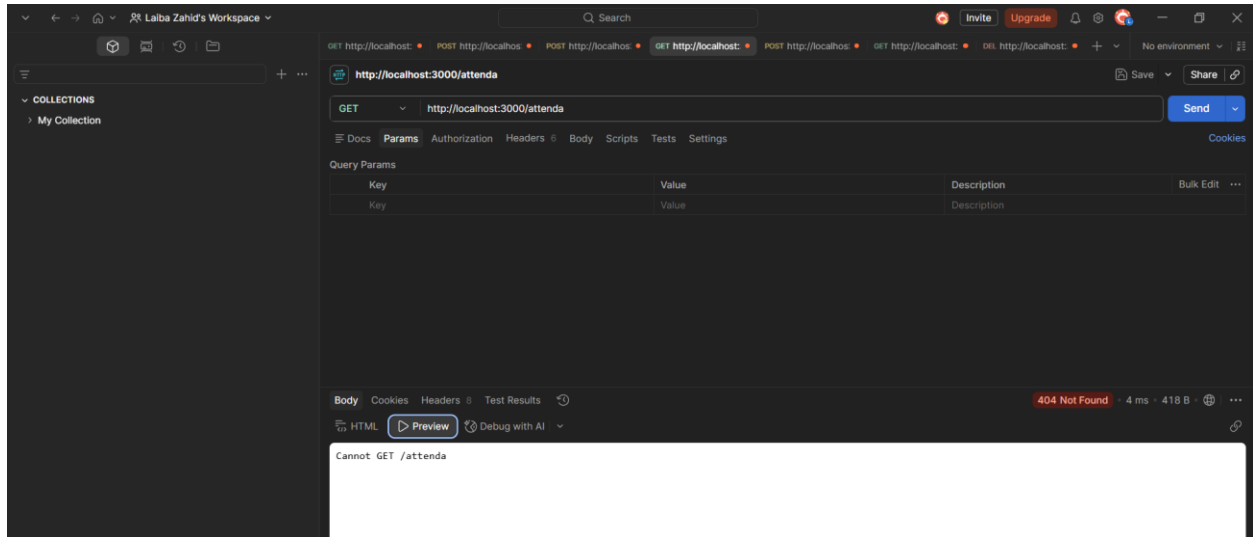
Screenshot 6 – POST Request Validation Error



Screenshot 7 – DELETE Request



Screenshot 8 – 404 Not Found Example



14. CHALLENGES AND SOLUTIONS

Challenge	Problem	Solution	Time
PowerShell Error	NPM scripts not running	Used Command Prompt	10 Minutes
Invalid Package Configuration	Corrupted package.json	Recreated project	15 Minutes
Module Not Found	Missing server.js	Created server.js file	5 Minutes
POST Request Error	JSON body not parsed	Added express.json() middleware	5 Minutes
Validation Logic	Complex validation checks	Implemented step-by-step validation	20 Minutes

15. API TESTING REPORT

15.1 Endpoint Testing

Endpoint	Method	Status	Response
/	GET	Pass	Welcome Message
/attendance	GET	Pass	All Students
/attendance/1	GET	Pass	Single Student
/attendance/999	GET	Pass	404 Not Found
/attendance	POST	Pass	201 Created
/attendance	POST	Pass	Validation Error
/attendance/3	DELETE	Pass	Success Response

15.2 Validation Testing

Test Case	Input	Expected Result	Status
Missing Name	No Name Field	400 Error	Pass
Missing Roll Number	No Roll Number	400 Error	Pass
Invalid Date	17-06-2026	400 Error	Pass
Invalid Status	pending	400 Error	Pass
Valid Data	Correct Input	201 Created	Pass

16. CONCLUSION AND LEARNINGS

The Student Attendance Management API was successfully developed and tested according to all project requirements.

The project demonstrates proficiency in backend API development using Node.js and Express.js while applying validation, error handling, routing, and testing concepts.

Key Learnings

1. REST API Development using Node.js and Express.js
2. API Endpoint Design and Implementation
3. HTTP Methods (GET, POST, DELETE)
4. Server-Side Data Validation
5. JSON Request and Response Handling
6. HTTP Status Code Management
7. Postman-Based API Testing
8. Error Handling using try-catch blocks

Future Improvements

- MongoDB Integration
 - JWT Authentication
 - User Registration Module
 - User Login Module
 - Frontend Interface Development
 - Cloud Deployment
-

18. GITHUB REPOSITORY

Repository Link

<https://github.com/lybah-gif/Decodeslab-project-2>

19. DECLARATION

I hereby declare that this project, "**Full Stack Project 2 – Backend API Development**," is my original work. All requirements specified in the Decodelabs Industrial Training Program project brief have been successfully completed.

The project was developed using:

- Node.js

- Express.js
- JavaScript (Vanilla)

No additional frameworks or libraries were used beyond those required for backend development.

The implementation follows industry-standard coding practices and internship guidelines.

Student Information

Student Information

Field	Details
Name	Laiba Zahid
Batch	2026
Project	Full Stack Project 2
Date	June 17, 2026

End of Document
